

Lab03 - Transparências em Sistemas Distribuídos e gRPC

Iniciar tarefa

- Vencimento Domingo por 23:59
- Pontos 5
- Enviando um URL de site

Roteiro de Laboratório - Transparências em Sistemas Distribuídos e gRPC

Disciplina: Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas

Unidade: U1 - Introdução ao Desenvolvimento de Aplicações Distribuídas

Professores: T1 - Cleiton Tavares Silva · T2 - Cristiano de Macedo Neto

Modalidade: Prática, em duplas ou individual (conforme orientação do professor em sala)

Pré-requisito: ter concluído o roteiro “Revisão de Redes de Computadores”

(TCP/UDP/Multicast/WebSocket) - este laboratório retoma esse código para comparação.

Nota de transparência (uso de IA): este roteiro foi diagramado e organizado com apoio do Claude (Anthropic), utilizado de forma responsável apenas para redação, estruturação e revisão do material. O aluno pode utilizar ferramentas de IA para apoiar rascunhos e revisões de código, **desde que declare o uso** na entrega e seja capaz de explicar e defender qualquer trecho entregue. Copiar e colar sem entender o funcionamento do código caracteriza uso não responsável e será tratado como falta de integridade acadêmica.

1. Objetivos

Ao final deste laboratório, o aluno deve ser capaz de:

- Identificar e explicar, com exemplos concretos, os principais tipos de **transparência em sistemas distribuídos** (acesso, localização, concorrência, replicação, falha, mobilidade, desempenho e escala);
- Comparar, na prática, o quanto de transparência é obtido “na mão” (sockets, Parte anterior) versus o quanto é obtido “de graça” ao usar um framework de RPC;
- Definir um contrato de serviço com **Protocol Buffers** (arquivo `.proto`) e gerar código cliente/servidor a partir dele;
- Implementar um serviço **gRPC** com uma chamada **unária** (requisição/resposta simples) e uma chamada com **streaming de servidor**;
- Comparar as mesmas soluções de RPC implementadas em **Java** e em **Python**;
- Continuar utilizando o **Git** de forma disciplinada, com commits pequenos, atômicos e bem descritos.

2. Tema do laboratório: Central de Atendimento da Turma via gRPC

Este laboratório reaproveita o cenário do roteiro anterior - a **central de comunicação da turma** - só que, desta vez, em vez de abrir sockets e montar mensagens de texto “na mão”, a comunicação é definida por um **contrato formal** (o arquivo `.proto`), e o código de rede é **gerado automaticamente** a partir dele.

Parte	Conteúdo	O que representa no cenário
A	Transparências em Sistemas Distribuídos	Revisão conceitual + comparação com o laboratório anterior
B	Protocol Buffers e o contrato do serviço	Definição de como o aluno “pergunta” e o monitor “responde”
C	RPC unário - <code>ConsultarHorario</code>	Aluno pergunta as horas ao monitor e recebe uma resposta - só que agora parece uma chamada de método comum
D	RPC com streaming - <code>AcompanharAvisos</code>	O mesmo mural de avisos em tempo real da Parte D do roteiro anterior, agora entregue via gRPC

3. Preparação do ambiente

Este roteiro assume **Windows**, usando o **PowerShell** como terminal padrão (mesma convenção do roteiro anterior).

Antes de começar, garanta que você tem instalado:

- **Java JDK 17 ou superior** (`java -version`)
- **Maven 3.8+** (`mvn -version`) - o Maven vai baixar automaticamente o compilador do Protocol Buffers (`protoc`) e o plugin de geração de código do gRPC, não é preciso instalá-los à parte
- **Python 3.10+**, com “Add python.exe to PATH” marcado na instalação (`python --version`)
- **Git for Windows** configurado (`git config --global user.name / user.email`)
- Um editor de sua preferência (VS Code, IntelliJ, PyCharm etc.)

Alerta do Firewall do Windows: como no roteiro anterior, na primeira execução de cada servidor o **Firewall do Windows Defender** deve pedir permissão de rede - clique em **“Permitir acesso”**.

3.1 Estrutura do repositório

```
lab-grpc/
├── proto/
│   └── central.proto
├── java/
│   └── grpc-central/
│       ├── pom.xml
│       └── src/main/
│           ├── java/br/pucminas/labdamd/central/
│           │   ├── ServidorCentral.java
│           │   └── ClienteCentral.java
│           └── proto/
```

```

├── central.proto
├── python/
│   ├── grpc_central/
│   │   ├── servidor_central.py
│   │   └── cliente_central.py
│   └── evidencias/
│       ├── transparencia/
│       ├── unario/
│       └── streaming/
├── RESPOSTAS.md
└── README.md

```

```

git add .
git commit -m "chore: estrutura inicial do repositório"

```

Todas as respostas às questões de cada parte devem ser escritas no arquivo **RESPOSTAS.md**, na raiz do repositório, organizadas por parte (A, B, C, D).

3.2 Evidências de teste (prints de tela)

Assim como no roteiro anterior: capture ao menos **um print de tela** por exemplo funcionando, mostrando execução real (não apenas código), com a saída do comando **Get-Date** visível em algum terminal do print para comprovar que é uma execução sua e recente.

- Parte A: print não se aplica (é uma parte conceitual) - a evidência dessa parte é a resposta em **RESPOSTAS.md**.
- Parte C (unário): **evidencias/unario/unario-java.png** e **evidencias/unario/unario-python.png**.
- Parte D (streaming): **evidencias/streaming/streaming-java.png** e **evidencias/streaming/streaming-python.png**.

3.3 Portas exclusivas (evite colisão com colegas)

Use o mesmo **OFFSET** pessoal calculado no roteiro anterior (dois últimos dígitos da sua matrícula/RA). Aqui ele se aplica às portas dos servidores gRPC:

Servidor	Porta-base	Sua porta
gRPC - Java	50051	50051 + OFFSET
gRPC - Python	50061	50061 + OFFSET

As bases já são diferentes entre Java e Python de propósito, para que você possa rodar os dois servidores ao mesmo tempo na sua própria máquina sem conflito, além de evitar colisão com colegas na mesma rede.

3.4 Trabalhando em dupla com Git

Vale a mesma orientação do roteiro anterior: cada integrante deve, sempre que possível, fazer seus próprios commits (usuário Git próprio, ou commits **Co-authored-by:** quando programarem juntos na

mesma máquina). Repositórios onde 100% dos commits pertencem a um só integrante podem gerar questionamento sobre a participação individual.

4. Parte A - Transparências em Sistemas Distribuídos

Conceito: um sistema distribuído é mais fácil de usar (e de programar) quanto mais ele consegue **esconder** do usuário e do programador os detalhes de que existem vários computadores, redes e falhas envolvidos. Cada tipo de “coisa escondida” tem um nome:

Transparência	O que ela esconde
Acesso	Diferenças na forma de representar dados e de acessar um recurso - local ou remoto, a chamada “parece” a mesma
Localização	Onde o recurso está fisicamente localizado
Concorrência	Que o recurso pode estar sendo usado por vários usuários/processos ao mesmo tempo
Replicação	Que o recurso pode existir em várias cópias
Falha	Que o recurso (ou parte da rede) pode falhar e se recuperar
Mobilidade	Que o recurso pode mudar de local sem que quem o usa perceba
Desempenho	Que o recurso pode ser realocado para melhorar o desempenho
Escala	Que o sistema pode crescer em tamanho sem que sua estrutura ou os aplicativos que o usam precisem mudar

Essa classificação vem do modelo de referência ODP (ISO/IEC 10746) e é amplamente usada em livros-texto de Sistemas Distribuídos (Coulouris et al.; Tanenbaum & Van Steen).

4.1 Tarefa (sem código novo - revisão do laboratório anterior)

Abra novamente o repositório do laboratório de **Revisão de Redes** (TCP, UDP, Multicast, WebSocket) e responda, em **RESPOSTAS.md**, para **cada uma** das quatro soluções (TCP, UDP, Multicast, WebSocket):

1. O endereço do servidor (**localhost**, IP, grupo multicast) está **escrito diretamente no código** do cliente? Isso favorece ou prejudica a transparência de localização?
2. Para “perguntar uma coisa” ao servidor, o cliente precisa montar uma **string de texto** manualmente (e o servidor precisa interpretá-la/fazer *parsing*)? Isso é meio-termo, presença ou ausência de transparência de acesso?
3. O que aconteceria com o cliente se o servidor mudasse de máquina amanhã? Alguma dessas quatro soluções sobreviveria a essa mudança sem alterar o código-fonte do cliente?

Depois de implementar as Partes C e D deste roteiro (gRPC), volte a este exercício e complemente sua resposta comparando com o que você observou usando RPC - essa comparação é justamente a Pergunta 3 da seção 4.3.

4.2 Commit desta parte

```
git add RESPOSTAS.md
git commit -m "docs(transparencia): responde reflexao sobre o lab anterior"
```

4.3 Perguntas - Parte A (responder em `RESPOSTAS.md`, junto com a Tarefa 4.1)

1. Dentre os 8 tipos de transparência listados, qual você diria que é a mais visível para o programador que está **usando** um serviço remoto (e não construindo a infraestrutura por trás dele)? Justifique.
2. Transparência total é sempre desejável? Dê um exemplo (pode ser hipotético) de uma situação em que esconder completamente que uma operação é remota atrapalharia mais do que ajudaria (dica: pense em desempenho ou em tratamento de falhas).
3. (*Responder depois de concluir as Partes C e D*) Comparando o cliente TCP do laboratório anterior com o cliente gRPC que você vai construir agora: qual dos dois exige que você “pense em rede” (sockets, `send` / `receive`, parsing de string) e qual permite que você “pense no problema” (chamar uma função e receber um resultado)? A que tipo de transparência isso se relaciona?

5. Parte B - Protocol Buffers e o contrato do serviço

Conceito: o **Protocol Buffers** (protobuf) é uma linguagem de definição de interface (IDL): você descreve, uma única vez, em um arquivo `.proto`, quais operações existem (o *serviço*) e qual o formato dos dados trocados (as *mensagens*). A partir desse único arquivo, ferramentas geram automaticamente código cliente e servidor em várias linguagens - inclusive a serialização/desserialização dos dados, que no laboratório anterior vocês fizeram à mão (`getBytes()`, `decode("utf-8")` etc.).

gRPC é um framework de RPC (Remote Procedure Call) que usa Protocol Buffers como formato de mensagens e HTTP/2 como transporte, oferecendo diferentes estilos de chamada - neste roteiro usaremos duas: **unária** (uma requisição, uma resposta) e **streaming de servidor** (uma requisição, várias respostas ao longo do tempo).

5.1 `proto/central.proto`

```
syntax = "proto3";

package central;

option java_package = "br.pucminas.labdamd.central";
option java_multiple_files = true;
option java_outer_classname = "CentralProto";

service CentralAtendimento {
    // RPC unário: uma pergunta, uma resposta - "parece" uma chamada de função comum
    rpc ConsultarHorario (PerguntaHorario) returns (RespostaHorario);
```

```
// RPC com streaming de servidor: uma inscrição, várias respostas ao longo do tempo
rpc AcompanharAvisos (InscricaoAvisos) returns (stream Aviso);
}

message PerguntaHorario {
    string nome_aluno = 1;
}

message RespostaHorario {
    string horario_atual = 1;
    string mensagem = 2;
}

message InscricaoAvisos {
    string nome_aluno = 1;
}

message Aviso {
    int32 numero = 1;
    string texto = 2;
}
```

Copie este arquivo também para `java/grpc-central/src/main/proto/central.proto` (o Maven espera encontrá-lo lá):

```
Copy-Item proto/central.proto java/grpc-central/src/main/proto/central.proto
```

5.2 Java - `java/grpc-central/pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>br.pucminas.labdamd</groupId>
  <artifactId>grpc-central</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <grpc.version>1.62.2</grpc.version>
    <protobuf.version>3.25.3</protobuf.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-netty-shaded</artifactId>
      <version>${grpc.version}</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-protobuf</artifactId>
      <version>${grpc.version}</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-stub</artifactId>
      <version>${grpc.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.tomcat</groupId>
      <artifactId>annotations-api</artifactId>
      <version>6.0.53</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
```

```

<build>
  <extensions>
    <extension>
      <groupId>kr.motd.maven</groupId>
      <artifactId>os-maven-plugin</artifactId>
      <version>1.7.1</version>
    </extension>
  </extensions>
  <plugins>
    <plugin>
      <groupId>org.xolstice.maven.plugins</groupId>
      <artifactId>protobuf-maven-plugin</artifactId>
      <version>0.6.1</version>
      <configuration>
        <protocArtifact>com.google.protobuf:protoc:${protobuf.version}:exe:${os.detected.classifier}</protocArtifact>
        <pluginId>grpc-java</pluginId>
        <pluginArtifact>io.grpc:protoc-gen-grpc-java:${grpc.version}:exe:${os.detected.classifier}</pluginArtifact>
      </configuration>
      <executions>
        <execution>
          <goals>
            <goal>compile</goal>
            <goal>compile-custom</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.1.0</version>
    </plugin>
  </plugins>
</build>
</project>

```

Gere o código a partir do `.proto` (o Maven baixa o `protoc` e o plugin do gRPC automaticamente na primeira execução - pode demorar um pouco):

```

cd java/grpc-central
mvn compile

```

Se o `mvn compile` terminar com `BUILD SUCCESS`, as classes `CentralAtendimentoGrpc`, `PerguntaHorario`, `RespostaHorario`, `InscricaoAvisos` e `Aviso` foram geradas em `target/generated-sources/` e já estão prontas para uso - você não precisa (nem deve) editá-las manualmente.

Se o Maven não conseguir baixar o `protoc` (proxy/firewall da rede): isso costuma acontecer com a mesma causa do problema já visto na Parte D do roteiro anterior (WebSocket). Consulte o professor sobre um espelho local dos pacotes ou sobre testar em outra rede - documente a tentativa em `RESPOSTAS.md` caso não consiga contornar.

5.3 Python - dependências

```

cd python/grpc_central
pip install grpcio grpcio-tools

```


Gere o código a partir do mesmo `.proto` (note o caminho relativo até `proto/central.proto`):

```
python -m grpc_tools.protoc -I ../../proto --python_out=. --grpc_python_out=. ../../proto/central.proto
```

Isso cria dois arquivos nesta pasta: `central_pb2.py` (as mensagens) e `central_pb2_grpc.py` (o serviço) - também não precisam ser editados manualmente. **Rode este comando de novo sempre que alterar o `.proto`.**

5.4 Commit desta parte

```
cd ../../
git add proto java/grpc-central/pom.xml java/grpc-central/src/main/proto python/grpc_central/central_pb2.py python/grpc_central/central_pb2_grpc.py
git commit -m "feat(proto): define contrato do servico CentralAtendimento e gera stubs"
```

5.5 Perguntas - Parte B

1. No laboratório anterior, cada um de vocês definiu o formato das mensagens de forma implícita (comentários e convenção entre quem escreveu o cliente e o servidor). Aqui, o formato está no `central.proto`. Qual a vantagem de ter esse contrato **explícito e gerado automaticamente** em vez de combinado apenas “de boca”?
2. O mesmo arquivo `central.proto` gerou código para Java e para Python. O que isso sugere sobre como equipes que usam linguagens diferentes podem se comunicar em um sistema distribuído real?
3. Observe os arquivos gerados (`target/generated-sources/.../CentralAtendimentoGrpc.java` ou `central_pb2_grpc.py`). Sem entender todo o código gerado, você consegue identificar onde ficam definidas as operações `ConsultarHorario` e `AcompanharAvisos` ? Cite o nome de pelo menos uma classe ou método gerado que você reconheceu.

6. Parte C - RPC unário: `ConsultarHorario`

Conceito: numa chamada **unária**, o cliente envia uma requisição e recebe exatamente uma resposta - é o estilo de RPC mais parecido com uma chamada de função/método comum, e é aqui que a **transparência de acesso** fica mais evidente: `stub.consultarHorario(pergunta)` parece uma chamada local, mas está, por baixo dos panos, serializando a mensagem, abrindo uma conexão HTTP/2, enviando pela rede, e desserializando a resposta.

6.1 Java - `ServidorCentral.java` (parte 1: unário)

Crie `java/grpc-central/src/main/java/br/pucminas/labdamd/central/ServidorCentral.java`:


```

package br.pucminas.labdamd.central;

import io.grpc.Server;
import io.grpc.ServerBuilder;
import io.grpc.stub.StreamObserver;

import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ServidorCentral {
    // TODO: substitua pelo seu OFFSET pessoal (ver seção 3.3)
    static final int OFFSET = 0;

    public static void main(String[] args) throws IOException, InterruptedException {
        int porta = 50051 + OFFSET;

        Server servidor = ServerBuilder.forPort(porta)
            .addService(new CentralAtendimentoImpl())
            .build();

        servidor.start();
        System.out.println("[gRPC] Servidor da Central ouvindo na porta " + porta);
        servidor.awaitTermination();
    }

    static class CentralAtendimentoImpl extends CentralAtendimentoGrpc.CentralAtendimentoImplBase {
        @Override
        public void consultarHorario(PerguntaHorario pedido, StreamObserver<RespostaHorario> observador) {
            String horario = LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"));
            System.out.println("[gRPC] ConsultarHorario chamado por: " + pedido.getNomeAluno());

            RespostaHorario resposta = RespostaHorario.newBuilder()
                .setHorarioAtual(horario)
                .setMensagem("Olá, " + pedido.getNomeAluno() + "! Agora são " + horario +
                    ".")
                .build();

            observador.onNext(resposta);
            observador.onCompleted();
        }
    }
}

```

Vamos completar esta classe com o método de streaming na Parte D - por enquanto, rode e teste só o `ConsultarHorario`.

6.2 Java - `ClienteCentral.java` (parte 1: unário)

Crie `java/grpc-central/src/main/java/br/pucminas/labdamd/central/ClienteCentral.java`:

```

package br.pucminas.labdamd.central;

import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;

import java.util.Scanner;

public class ClienteCentral {
    // TODO: substitua pelo seu OFFSET pessoal - use o MESMO valor do servidor
    static final int OFFSET = 0;

    public static void main(String[] args) {

```

```

    int porta = 50051 + OFFSET;

    ManagedChannel canal = ManagedChannelBuilder.forAddress("localhost", porta)
        .usePlaintext()
        .build();

    try {
        CentralAtendimentoGrpc.CentralAtendimentoBlockingStub stub =
            CentralAtendimentoGrpc.newBlockingStub(canal);

        Scanner teclado = new Scanner(System.in);
        System.out.print("Digite seu nome: ");
        String nome = teclado.nextLine();

        // Chamada unária: parece uma chamada de método local, mas atravessa a rede
        PerguntaHorario pergunta = PerguntaHorario.newBuilder().setNomeAluno(nome).build();
        RespostaHorario resposta = stub.consultarHorario(pergunta);
        System.out.println("[gRPC] " + resposta.getMensagem());
    } finally {
        canal.shutdown();
    }
}
}

```

Como executar:

```

cd java/grpc-central
mvn compile exec:java -Dexec.mainClass=br.pucminas.labdamd.central.ServidorCentral      # em um t
erminal
mvn compile exec:java -Dexec.mainClass=br.pucminas.labdamd.central.ClienteCentral      # em outr
o terminal

```

6.3 Python - `servidor_central.py` (parte 1: unário)

Crie `python/grpc_central/servidor_central.py`:

```

from concurrent import futures
from datetime import datetime

import grpc

import central_pb2
import central_pb2_grpc

# TODO: substitua pelo seu OFFSET pessoal (ver seção 3.3)
OFFSET = 0
PORTA = 50061 + OFFSET

class CentralAtendimentoServicer(central_pb2_grpc.CentralAtendimentoServicer):

    def ConsultarHorario(self, request, context):
        horario = datetime.now().strftime("%H:%M:%S")
        print(f"[gRPC] ConsultarHorario chamado por: {request.nome_aluno}")
        return central_pb2.RespostaHorario(
            horario_atual=horario,
            mensagem=f"Olá, {request.nome_aluno}! Agora são {horario}.",
        )

def main():
    servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    central_pb2_grpc.add_CentralAtendimentoServicer_to_server(CentralAtendimentoServicer(), servi
dor)
    servidor.add_insecure_port(f"[::]:{PORTA}")
    servidor.start()

```

```
print(f"[gRPC] Servidor da Central ouvindo na porta {PORTA}")
servidor.wait_for_termination()
```

```
if __name__ == "__main__":
    main()
```

Vamos completar esta classe com o método de streaming na Parte D.

6.4 Python - `cliente_central.py` (parte 1: unário)

Crie `python/grpc_central/cliente_central.py`:

```
import grpc

import central_pb2
import central_pb2_grpc

# TODO: substitua pelo seu OFFSET pessoal - use o MESMO valor do servidor
OFFSET = 0
PORTA = 50061 + OFFSET

def main():
    canal = grpc.insecure_channel(f"localhost:{PORTA}")
    stub = central_pb2_grpc.CentralAtendimentoStub(canal)

    nome = input("Digite seu nome: ")

    # Chamada unária: parece uma chamada de função local, mas atravessa a rede
    resposta = stub.ConsultarHorario(central_pb2.PerguntaHorario(nome_aluno=nome))
    print(f"[gRPC] {resposta.mensagem}")

if __name__ == "__main__":
    main()
```

Como executar:

```
cd python/grpc_central
python servidor_central.py      # em um terminal
python cliente_central.py      # em outro terminal
```

6.5 Tarefa

1. Rode o servidor e o cliente em Java e confirme que a resposta com o horário chega corretamente.
2. Rode o servidor e o cliente em Python e confirme o mesmo.
3. Capture um print de tela para cada linguagem, mostrando o terminal do servidor e do cliente com a chamada `ConsultarHorario` completa, e salve-os como `evidencias/unario/unario-java.png` e `evidencias/unario/unario-python.png`.
4. Faça o commit:

```
git add java/grpc-central/src python/grpc_central/servidor_central.py python/grpc_central/cliente_central.py evidencias/unario
git commit -m "feat(unario): implementa RPC ConsultarHorario em Java e Python"
```

6.6 Perguntas - Parte C

1. No cliente, a linha `stub.consultarHorario(pergunta)` (Java) ou `stub.ConsultarHorario(...)` (Python) parece uma chamada de método comum. Cite, em alto nível, pelo menos três coisas que acontecem “por baixo dos panos” entre essa chamada e o `return` da função no servidor.
2. Compare esta implementação com o `ClienteTCP` do roteiro anterior. Onde estava, no TCP, o equivalente a “montar a mensagem” e “interpretar a resposta”? Quem faz esse trabalho agora, no gRPC?
3. O que aconteceria se você chamasse `stub.consultarHorario(pergunta)` com o servidor desligado? Teste e descreva o comportamento observado (em qualquer uma das duas linguagens).

7. Parte D - RPC com streaming de servidor: `AcompanharAvisos`

Conceito: em uma chamada de **streaming de servidor**, o cliente faz uma única requisição, mas o servidor pode responder com **vários** valores ao longo do tempo, em uma única conexão - é o equivalente, em RPC, ao que vocês fizeram “na mão” com Multicast e WebSocket no roteiro anterior.

7.1 Java - completando `ServidorCentral.java`

Adicione o método `acompanharAvisos` dentro da classe `CentralAtendimentoImpl` (mesmo arquivo da Parte C):

```
@Override
public void acompanharAvisos(InscricaoAvisos pedido, StreamObserver<Aviso> observador) {
    System.out.println("[gRPC] AcompanharAvisos: " + pedido.getNomeAluno() + " se inscrev
eu.");
    try {
        for (int i = 1; i <= 5; i++) {
            Aviso aviso = Aviso.newBuilder()
                .setNumero(i)
                .setTexto("Aviso #" + i + ": a aula começa em " + (5 - i) + " minuto
(s)!")
                .build();
            observador.onNext(aviso);
            Thread.sleep(2000);
        }
        observador.onCompleted();
    } catch (InterruptedException e) {
        observador.onError(e);
    }
}
```

7.2 Java - completando `ClienteCentral.java`

Adicione, no `main` do `ClienteCentral.java` (após a chamada de `ConsultarHorario`, antes do `finally`):

```
// Chamada com streaming: o servidor envia vários Avisos ao longo do tempo
System.out.println("[gRPC] Inscrevendo-se para acompanhar avisos...");
```

```
InscricaoAvisos inscricao = InscricaoAvisos.newBuilder().setNomeAluno(nome).build();
java.util.Iterator<Aviso> avisos = stub.acompanharAvisos(inscricao);
while (avisos.hasNext()) {
    Aviso aviso = avisos.next();
    System.out.println("[gRPC] Recebido: " + aviso.getTexto());
}
```

7.3 Python - completando `servidor_central.py`

Adicione o método `AcompanharAvisos` dentro da classe `CentralAtendimentoServicer` (mesmo arquivo da Parte C) - note que, em Python, um método de streaming é simplesmente uma função geradora, usando `yield` em vez de `return`:

```
def AcompanharAvisos(self, request, context):
    print(f"[gRPC] AcompanharAvisos: {request.nome_aluno} se inscreveu.")
    for i in range(1, 6):
        yield central_pb2.Aviso(
            numero=i,
            texto=f"Aviso #{i}: a aula começa em {5 - i} minuto(s)!",
        )
    time.sleep(2)
```

Não esqueça de adicionar `import time` no topo do arquivo.

7.4 Python - completando `cliente_central.py`

Adicione, na função `main()` do `cliente_central.py` (após a chamada de `ConsultarHorario`):

```
# Chamada com streaming: o servidor envia vários Avisos ao longo do tempo
print("[gRPC] Inscrevendo-se para acompanhar avisos...")
for aviso in stub.AcompanharAvisos(central_pb2.InscricaoAvisos(nome_aluno=nome)):
    print(f"[gRPC] Recebido: {aviso.texto}")
```

7.5 Tarefa

1. Rode novamente o servidor e o cliente em Java, agora observando também os avisos chegando via streaming (5 avisos, a cada 2 segundos).
2. Repita em Python.
3. Capture um print de tela para cada linguagem, mostrando pelo menos 2-3 avisos recebidos em sequência, e salve-os como `evidencias/streaming/streaming-java.png` e `evidencias/streaming/streaming-python.png`.
4. Volte à Pergunta 3 da seção 4.3 (Parte A) e complete sua resposta agora que já implementou o RPC com streaming.
5. Faça o commit:

```
git add java/grpc-central/src python/grpc_central evidencias/streaming RESPOSTAS.md
git commit -m "feat(streaming): implementa RPC AcompanharAvisos em Java e Python"
```

7.6 Perguntas - Parte D

1. No laboratório anterior, o Multicast usava um endereço de grupo (`230.0.0.1`) para alcançar vários clientes com um único envio; aqui, o streaming gRPC é **um servidor conversando com um cliente por vez**, só que ao longo de uma conexão só. Se você quisesse que **vários** clientes gRPC recebessem os mesmos avisos ao mesmo tempo, o que precisaria mudar na implementação do servidor?
2. Compare o método de streaming em Java (`StreamObserver` , chamando `onNext()` repetidamente) com o de Python (uma função geradora usando `yield`). Os dois alcançam o mesmo resultado - qual das duas abordagens você achou mais natural de entender? Justifique.
3. No método `acompanharAvisos` / `AcompanharAvisos` , o que aconteceria se o cliente fechasse a conexão (por exemplo, fechando o terminal) no meio do envio dos 5 avisos? Pesquise ou teste o comportamento e descreva o que observou.

8. Checklist de entrega

- Repositório Git com a estrutura de pastas indicada, hospedado conforme orientação do professor
- Ao menos **4 commits principais** (um por parte - Transparências, Proto/setup, RPC unário, RPC streaming), com mensagens claras no padrão `tipo(escopo): descrição`
- Histórico de commits **incremental** - commits pequenos ao longo do desenvolvimento contam a favor
- `central.proto` compilando e gerando stubs corretamente para Java e Python
- As duas soluções de RPC (unário e streaming), cada uma em **Java e Python**, executando corretamente
- Pasta `evidencias/` com **4 prints de tela** (unário e streaming, em Java e Python), conforme indicado em cada tarefa
- Arquivo `RESPOSTAS.md` completo: a reflexão da Parte A (seção 4.1) + as 12 perguntas (3 por parte) respondidas de forma própria e fundamentada, incluindo a Pergunta 3 da Parte A respondida **depois** de concluir C e D

9. Critérios de avaliação

Critério	O que é observado
Commits	Existência, granularidade e clareza das mensagens de commit ao longo de todo o desenvolvimento
Funcionamento do código	O <code>.proto</code> compila e as 4 soluções (2 estilos de RPC × 2 linguagens) executam e realizam a comunicação esperada
Evidências de teste	Presença, na pasta <code>evidencias/</code> , dos 4 prints de tela exigidos, mostrando execução real
Respostas às questões	Compreensão demonstrada nas respostas de <code>RESPOSTAS.md</code> - em especial a capacidade de comparar o RPC com o que foi feito “na mão” no laboratório anterior, e não apenas definições copiadas de fontes externas

A ponderação exata entre esses critérios é definida pelo professor responsável pela turma (T1 ou T2), conforme os critérios de avaliação apresentados em sala.

10. Referências

- COULOURIS, George et al. *Distributed Systems: Concepts and Design*. 5th ed. Addison-Wesley, 2011. (Capítulo sobre transparência em sistemas distribuídos.)
- TANENBAUM, A. S.; VAN STEEN, M. *Sistemas Distribuídos: Princípios e Paradigmas*. Tradução da 2ª edição. Pearson, 2007.
- ISO/IEC 10746 - *Reference Model of Open Distributed Processing (RM-ODP)*.
- gRPC Authors. *gRPC Documentation*. Disponível em: <https://grpc.io/docs/> 
(<https://grpc.io/docs/>)
- gRPC Authors. *Protocol Buffers Documentation*. Disponível em: <https://protobuf.dev/> 
(<https://protobuf.dev/>)
- gRPC Authors. *Java Quick Start*. Disponível em: <https://grpc.io/docs/languages/java/quickstart/> 
(<https://grpc.io/docs/languages/java/quickstart/>)
- gRPC Authors. *Python Quick Start*. Disponível em: <https://grpc.io/docs/languages/python/quickstart/> 
(<https://grpc.io/docs/languages/python/quickstart/>)